optimal values.

$$k = \frac{-\log_e(p)}{\log_e(2)}$$

$$m = \frac{-n \log_e(p)}{(\log_e(2))^2}$$

Experimental results

The Bloom filter technique is used by Web browsers to filter malicious URLs. It helps to solve the problem of fast lookup from a list of such URLs. A browser has to get the information from a remote server, without a local lookup solution, which is slow and expensive. One solution is to use a HashTable to store these URLs. But HashTables have large memory requirements for huge lists such as those containing 10 million entries, which need 250MB to 300MB of memory.

A Bloom filter effectively handles such problems and offers constant time O(1) operations while using negligible memory. A Bloom filter of a size of one million entries would need only 1MB of memory. An input URL is checked in the Bloom filter embedded in the browser. If the filter returns *False*, then one can be sure that the URL is not present in the list of malicious sites, since the false negative case is not possible in a Bloom filter. If the filter returns *True*, the input URL could or could not be present in the malicious URL list. So the browser has to make a remote call to determine the validity of the URL.

The Bloom filter is very useful especially when the input search is a spurious value because the filter will identify it as 'not present' in the target set. This reduces the load of remote calls and decreases latency of the browser lookup operations.

We ran an experiment in which there was a list of one million malicious URLs generated randomly. The incoming URL was checked against the malicious URL list using a Bloom filter. We found that the Bloom filter successfully identified 88 per cent of malicious URLs. The code for the Bloom filter implementation and the test experiment is available at GitHub (*https://github.com/souravchk/Algorithms-and-Data-Structures/tree/master/bloom-filters*). END

References

[1] *https://bugra.github.io/work/notes/2016-06-05/a-gentle-introduction-to-bloom-filter/*

[2] *https://wiki.squid-cache.org/SquidFaq/CacheDigests*

By: Vishal Kanaujia and Sourav Chakraborty

Vishal Kanaujia is a software developer and open source software enthusiast. He works in a software company based in Bengaluru. He can be contacted via his website *http://freethreads.weebly.com/*.

Sourav Chakraborty is a software engineer by day and a dreamer by night. He is a hard core computerphile, numberphile and bibliophile. He is available at *https://souravchk.github.io*.